Flow Matching and Stochastic Interpolants

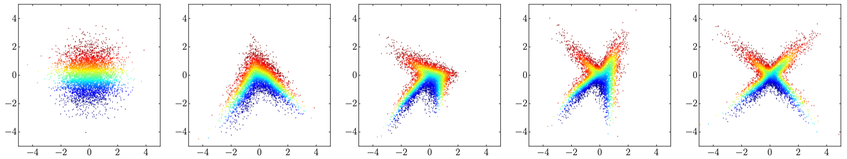
Gavin Kerrigan
2026 November 12

gavin.kerrigan@stats.ox.ac.uk
GavinKerrigan.github.io



Recap: Normalising Flows and Diffusion

Normalising Flows:



Only define one “direction” of transformation (noise to data, or data to noise)

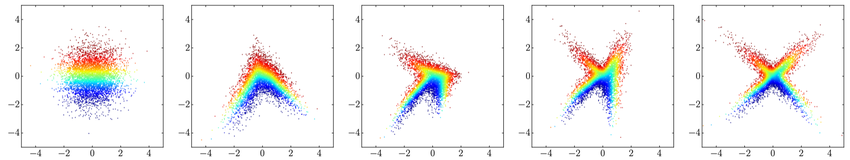
Trained by maximum likelihood — requires evaluating model $\log p_\theta(x)$

Discrete-Time models: require tractable architectures, easy-to-evaluate Jacobians

Continuous-Time models: re-cast generation process as a neural ODE

Recap: Normalising Flows and Diffusion

Normalising Flows:



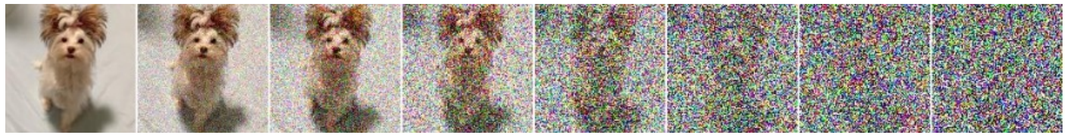
Only define one “direction” of transformation (noise to data, or data to noise)

Trained by maximum likelihood — requires evaluating model $\log p_\theta(x)$

Discrete-Time models: require tractable architectures, easy-to-evaluate Jacobians

Continuous-Time models: re-cast generation process as a neural ODE

Diffusion:



“Forward” process (data to noise) is fixed, not learned

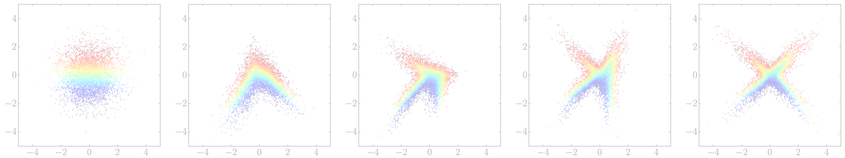
Generate samples by learning the time-reversal of this process

Recall DDPM: generation process happens in discrete time steps

Trained using a regression-based objective; coming from variational inference

Recap: Normalising Flows and Diffusion

Normalising Flows:



Only define one “direction” of transformation (noise to data, or data to noise)

Trained by maxi

Discrete-Time

Continuous-Ti

Can we somehow unify these two perspectives?

Continuous version of diffusion?

Flows without likelihood-based training?

Diffusion:



“Forward” process (data to noise) is fixed, not learned

Generate samples by learning the time-reversal of this process

Recall DDPM: generation process happens in discrete time steps

Trained using a regression-based objective; coming from variational inference

Can we somehow unify these two perspectives?

Continuous version of diffusion?

Yes: using stochastic differential equations (Song et al., 2021)

Modern diffusion approaches all based on SDEs

Can we somehow unify these two perspectives?

Continuous version of diffusion?

Yes: using stochastic differential equations (Song et al., 2021)

Modern diffusion approaches all based on SDEs

Flows without likelihood-based training?

Yes: a few closely related techniques

Stochastic Interpolants (Albergo et al., 2023)

Flow Matching (Lipman et al., 2022)

Rectified Flows (Liu et al., 2022)

Stochastic Differential Equations

Recall that in a continuous-time normalising flow:

Velocity field $v_\theta(x_t, t)$ implemented by a neural network

Sampling: draw $x_0 \sim \mathcal{N}(0, I)$, solve $dx_t = v_\theta(x_t, t)dt$

Euler scheme: $x_{t+h} = x_t + hv_\theta(x_t, t)$

Stochastic Differential Equations

Recall that in a continuous-time normalising flow:

Velocity field $v_\theta(x_t, t)$ implemented by a neural network

Sampling: draw $x_0 \sim \mathcal{N}(0, I)$, solve $dx_t = v_\theta(x_t, t)dt$

Euler scheme: $x_{t+h} = x_t + hv_\theta(x_t, t)$

Let us consider stochastic updates

Euler-Maruyama Scheme: $x_{t+h} = x_t + hv_\theta(x_t, t) + g(x_t, t)\sqrt{h}\epsilon \qquad \epsilon \sim \mathcal{N}(0, I)$

Adds a "jitter" to our velocity field — variance grows with the step size h

The function $g(x_t, t)$ allows for non-isotropic noise

Stochastic Differential Equations

Recall that in a continuous-time normalising flow:

Velocity field $v_\theta(x_t, t)$ implemented by a neural network

Sampling: draw $x_0 \sim \mathcal{N}(0, I)$, solve $dx_t = v_\theta(x_t, t)dt$

Euler scheme: $x_{t+h} = x_t + hv_\theta(x_t, t)$

Let us consider stochastic updates

Euler-Maruyama Scheme: $x_{t+h} = x_t + hv_\theta(x_t, t) + g(x_t, t)\sqrt{h}\epsilon \qquad \epsilon \sim \mathcal{N}(0, I)$

Adds a "jitter" to our velocity field — variance grows with the step size h

The function $g(x_t, t)$ allows for non-isotropic noise

Euler-Maruyama is a time discretisation of the stochastic differential equation

$$dx_t = v_\theta(x_t, t)dt + g(x_t, t)dW_t \qquad x_0 \sim \mathcal{N}(0, I)$$

W_t is a Brownian motion — a stochastic process

Rich theory behind SDEs — for now, just think of these a noisy ODE

Continuous-Time Diffusion: Forward Process

Drift Diffusion

$$dX_t = \mu(t, X_t)dt + \sigma(t, X_t)dW_t \qquad X_0 \sim p_{\text{data}}(x_0)$$

$$\mu : [0,T] \times \mathbb{R}^d \to \mathbb{R}^d \qquad \sigma : [0,T] \times \mathbb{R}^d \to \mathbb{R}$$

Example — Ornstein-Uhlenbeck Process

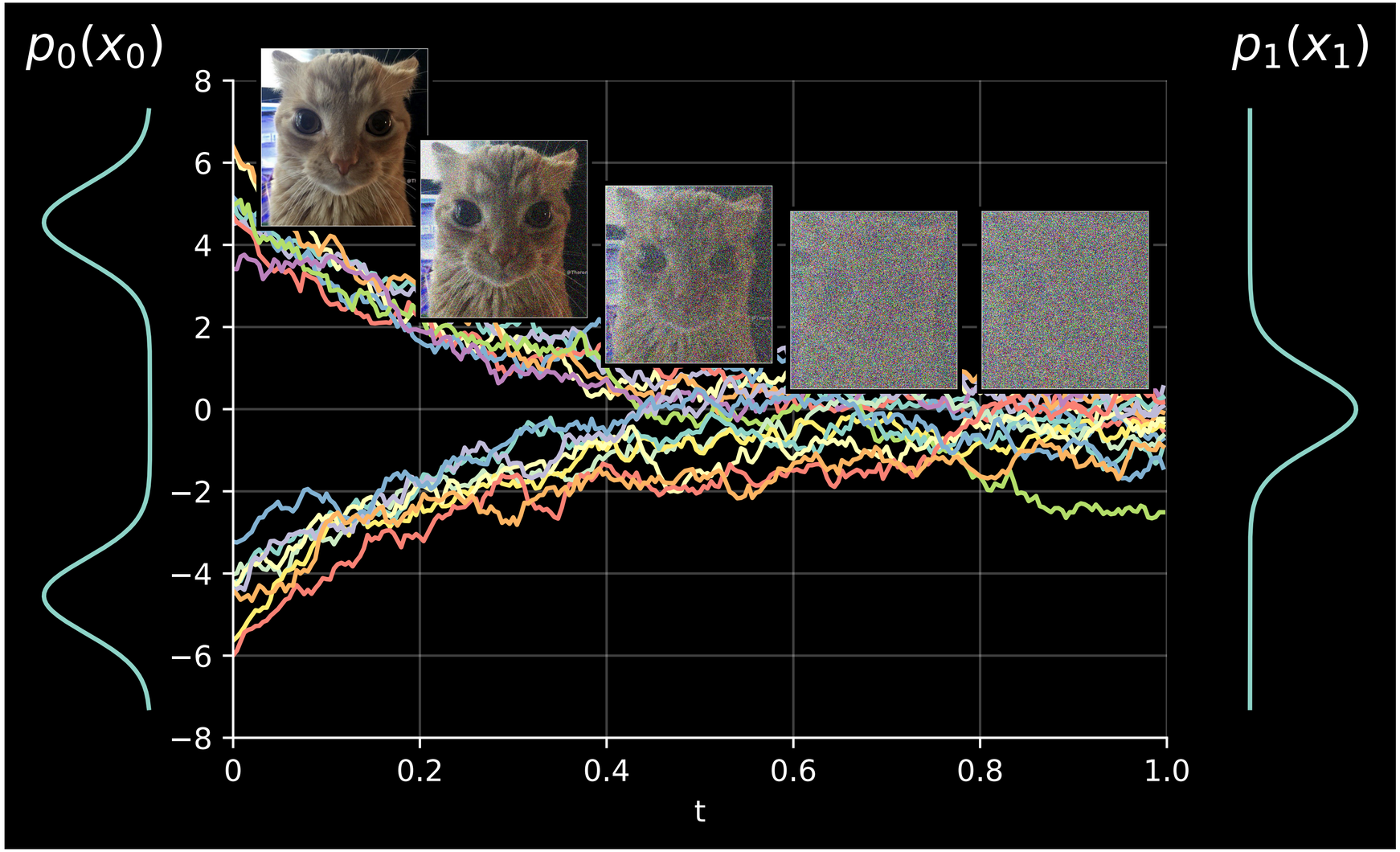
Noise Schedule

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)}dW_t \qquad X_0 \sim p_{\text{data}}(x_0)$$

$$\mu(t, X_t) = -\frac{1}{2}\beta(t)X_t \qquad \sigma(t, X_t) = \sqrt{\beta(t)}$$

Continuous-Time Diffusion: Forward Process

Example — Ornstein-Uhlenbeck Process $\mu(t, X_t) = -\frac{1}{2}\beta(t)X_t$ $\sigma(t, X_t) = \sqrt{\beta(t)}$



Forward Process

Example — Ornstein-Uhlenbeck Process

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)}dW_t \qquad X_0 \sim p_{\text{data}}(x_0)$$

Conditional densities can be given in closed-form:

$$p_t(x_t \mid x_0) = \mathcal{N}\left(x_0 \exp\left(-\frac{1}{2}\int_0^t \beta(s)ds\right), \mathbf{I} - \exp\left(-\int_0^t \beta(s)ds\right)\mathbf{I}\right)$$

… but the marginals are intractable

$$p_t(x_t) = \int p_t(x_t \mid x_0)p(x_0)dx_0$$

Forward Process

Forward Process: Example — Ornstein-Uhlenbeck Process

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)}dW_t \qquad X_0 \sim p_{\text{data}}(x_0)$$

$$p_t(x_t \mid x_0) = \mathcal{N}\left(x_0 \exp\left(-\frac{1}{2}\int_0^t \beta(s)ds\right), \mathbf{I} - \exp\left(-\int_0^t \beta(s)ds\right)\mathbf{I}\right)$$

As long as $\int_0^T \beta(s)ds$ is large,

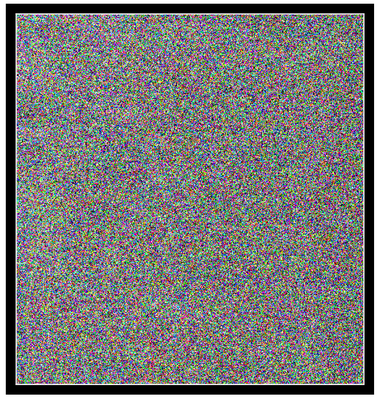
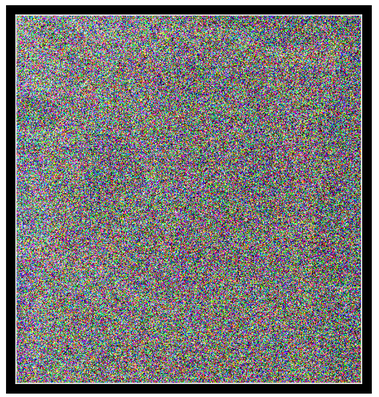
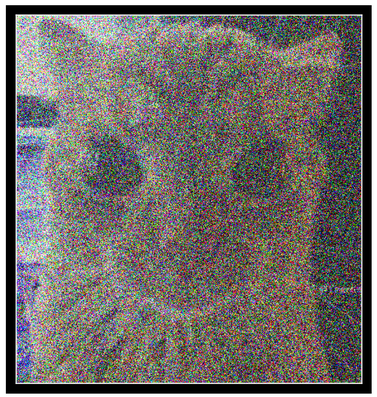
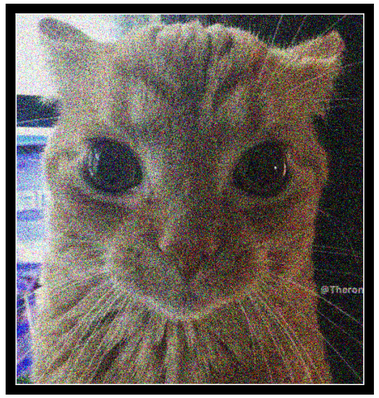
$$p(x_T) = \int p(x_T \mid x_0)p(x_0)dx_0 \approx \mathcal{N}(0, \mathbf{I})$$

Diffusion Generative Models

What if I could run my process *backwards in time?*

$$\overleftarrow{X}_t = X_{T-t}$$

$x_T \sim \mathcal{N}(0,I)$


$x_0 \sim p_{\text{data}}$

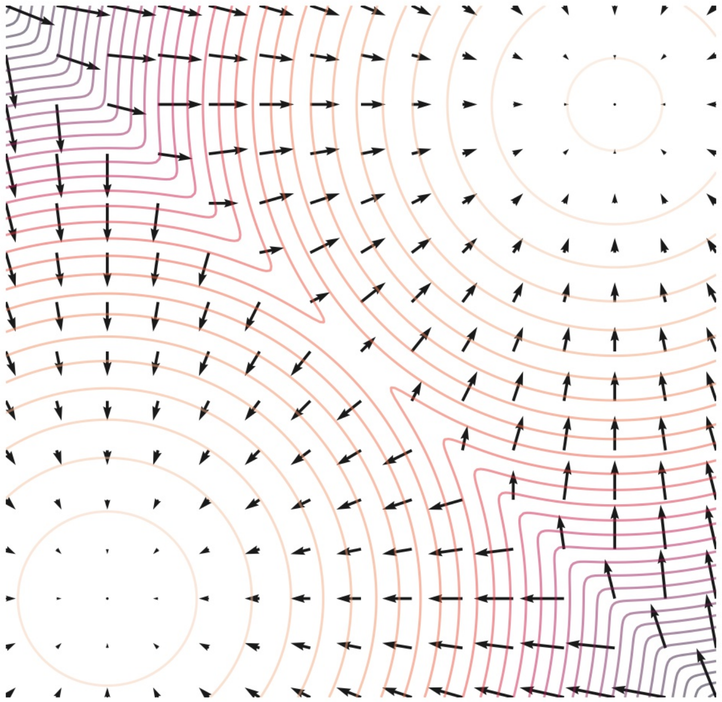
Reverse Process

$$dX_t = \mu(t, X_t)dt + \sigma(t, X_t)dW_t \qquad \overleftarrow{X}_t = X_{T-t}$$

Theorem (informal) [Anderson 1982, Haussman and Pardoux, 1986]:

$$d\overleftarrow{X}_t = \left[\mu(t, X_t) - \sigma^2(t)\,\nabla_x \log p_t(X_t)\right] dt + \sigma(t, X_t)d\overleftarrow{W}_t \qquad X_T \sim p_T(x_T)$$

(Stein) score function



[Song 2021]

Diffusion Generative Models

Reverse Process: $dX_t = \mu(t, X_t)dt + \sigma(t, X_t)dW_t \qquad \overleftarrow{X}_t = X_{T-t}$

Theorem (informal) [Anderson 1982, Haussman and Pardoux, 1986]:

$$d\overleftarrow{X}_t = \left[\mu(t, X_t) - \sigma^2(t)\nabla_x \log p_t(X_t)\right] dt + \sigma(t, X_t)d\overleftarrow{W}_t \qquad X_T \sim p_T(x_T)$$

(Stein) score function

Problem: score function is intractable

$$\nabla_{x_t} \log p_t(x_t) = \int \nabla_{x_t} \log p_t(x_t \mid x_0) p(x_0) dx_0$$

Score Matching

How can we model the scores?

$$\nabla_x \log p_t(x_t) \approx s_\theta(t, x_t)$$

$$\mathscr{L} = \mathbb{E}_{t, x_t} \left[\; || \nabla_x \log p_t(x_t) - s_\theta(t, x_t) ||^2 \; \right]$$

This is still intractable! Depends on the unknown scores.

Recall: we know $p_t(x_t \mid x_0)$

$\Longrightarrow$ know $\nabla_x \log p_t(x_t \mid x_0)$ analytically

$$\mathscr{L} = \mathbb{E}_{t, x_0, x_t} \left[\; || \nabla_x \log p_t(x_t \mid x_0) - s_\theta(t, x_t) ||^2 \; \right] + C$$

Score Matching

$$\mathscr{L} = \mathbb{E}_{t, x_0, x_t} \left[\lambda(t) || \nabla_x \log p_t(x_t \mid x_0) - s_\theta(t, x_t) ||^2 \right]$$

(Optional) loss weighting

Training Pseudocode

while not done training:

1. Sample a datapoint $x_0 \sim p_{\text{data}}$
2. Sample a time $t \sim \text{Unif}[0, T]$
3. Sample $x_t \sim p_t(x_t \mid x_0)$ (recall: closed-form & Gaussian)
4. Evaluate $\nabla_x \log p_t(x_t \mid x_0)$ (recall: closed-form)
5. Gradient descent step on $\lambda(t) || \nabla_{x_t} \log p_t(x_t \mid x_0) - s_\theta(t, x_t) ||_2^2$

Sampling

Given a score function, how do we draw samples?

True reverse process:

$$d\overleftarrow{X}_t = \left[\mu(t, X_t) - \sigma^2(t)\nabla_x \log p_t(X_t)\right] dt + \sigma(t, X_t) d\overleftarrow{W}_t \qquad X_T \sim p_T(x_T)$$

Approximate reverse process:

$$d\overleftarrow{X}_t = \left[\mu(t, X_t) - \sigma^2(t) s_\theta(t, X_t)\right] dt + \sigma(t, X_t) d\overleftarrow{W}_t \qquad X_T \sim \mathcal{N}(0, I)$$

Sampling

Approximate reverse process:

$$d\overleftarrow{X}_t = \left[\mu(t, X_t) - \sigma^2(t) s_\theta(t, X_t)\right] dt + \sigma(t, X_t) d\overleftarrow{W}_t \qquad X_T \sim \mathcal{N}(0, I)$$

Solve SDE numerically to sample

E.g. Euler-Maruyama Scheme:

$$T = t_0 > t_1 > \ldots > t_N = 0 \qquad \Delta t = T/N$$

$$X_T \sim \mathcal{N}(0, I)$$

For $t = t_0, t_1, \ldots, t_{N-1}$:

$$Z \sim \mathcal{N}(0, I)$$

$$X_{t-\Delta t} = X_t + \left[\mu(t, X_t) - \sigma^2(t) s_\theta(t, X_t)\right] \Delta t + \sigma(t)\sqrt{\Delta t} Z$$

Expensive! Often requires ~100s-1000s of steps

Each step requires an evaluation of model s_θ

Diffusion Generative Models

CelebA-HQ @ 1024x1024



[Song et al., 2021]

Questions?

Can we somehow unify these two perspectives?

Continuous version of diffusion?

Yes: using stochastic differential equations (Song et al., 2021)

Modern diffusion approaches all based on SDEs

Flows without likelihood-based training?

Yes: a few closely related techniques

Stochastic Interpolants (Albergo et al., 2023)

Flow Matching (Lipman et al., 2022)

Rectified Flows (Liu et al., 2022)

What's Wrong with Normalising Flows?

Proposition [Continuity Equation].

If the particles X_t follow the dynamics $dX_t = v(X_t, t)dt \qquad X_0 \sim p_0$

Then the density $p_t(x)$ at any time $t > 0$ solves the continuity PDE

$$\partial_t p_t(x) + \mathrm{div}(p_t(x)v(x,t)) = 0$$

What's Wrong with Normalising Flows?

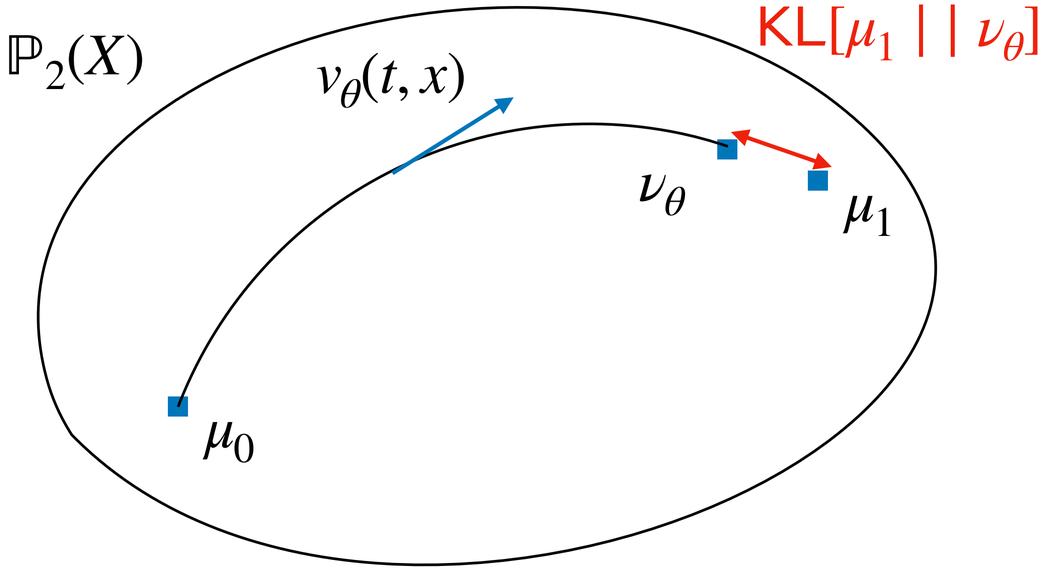
Proposition [Continuity Equation].

If the particles X_t follow the dynamics $dX_t = v(X_t, t)dt \qquad X_0 \sim p_0$

Then the density $p_t(x)$ at any time $t > 0$ solves the continuity PDE

$$\partial_t p_t(x) + \text{div}(p_t(x)v(x,t)) = 0$$

1. Fix μ_0
2. Write down some parametric vector field $v_\theta(t, x)$
3. Compute $\nu_\theta = [\phi_1]_\# \mu_0$
4. Minimize $\text{KL}[\mu_1 \,||\, \nu_\theta]$



What's Wrong with Normalising Flows?

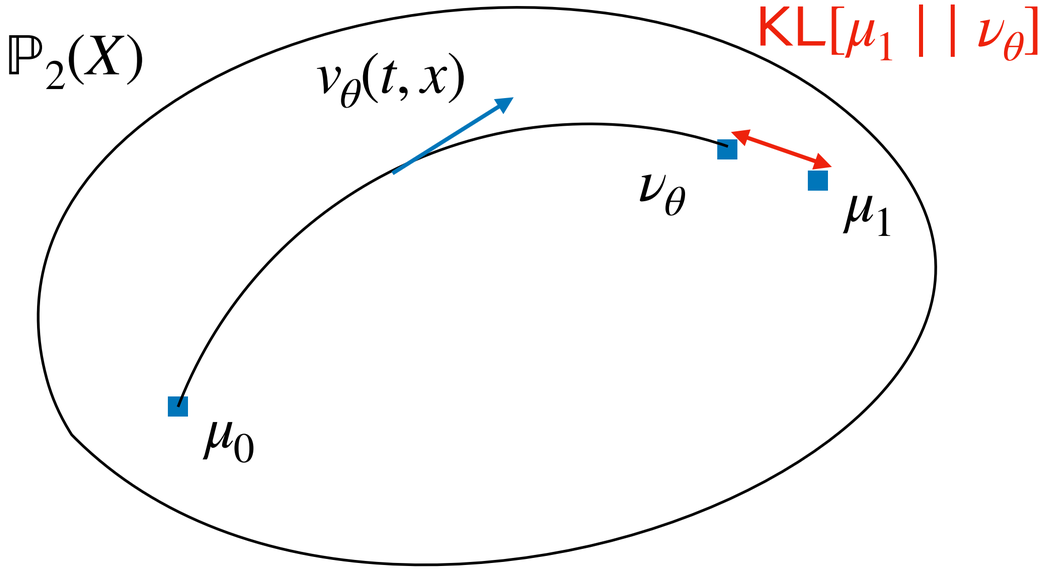
Proposition [Continuity Equation].

If the particles X_t follow the dynamics $dX_t = v(X_t, t)dt \qquad X_0 \sim p_0$

Then the density $p_t(x)$ at any time $t > 0$ solves the continuity PDE

$$\partial_t p_t(x) + \text{div}(p_t(x)v(x,t)) = 0$$

1. Fix μ_0
2. Write down some parametric vector field $v_\theta(t, x)$
3. Compute $\nu_\theta = [\phi_1]_\# \mu_0$
4. Minimize KL$[\mu_1 \,||\, \nu_\theta]$



Drawbacks:

1. Requires simulating ODE during training
2. Model has to do the hard work of finding a path
   - Hard to enforce a "nice" path

Flow Matching

Flow Matching (Lipman 2022): ODE-based framework that does not require simulations

Consider $t \in [0,1]$ and let $p_0(x_0) = \mathcal{N}(0,I)$

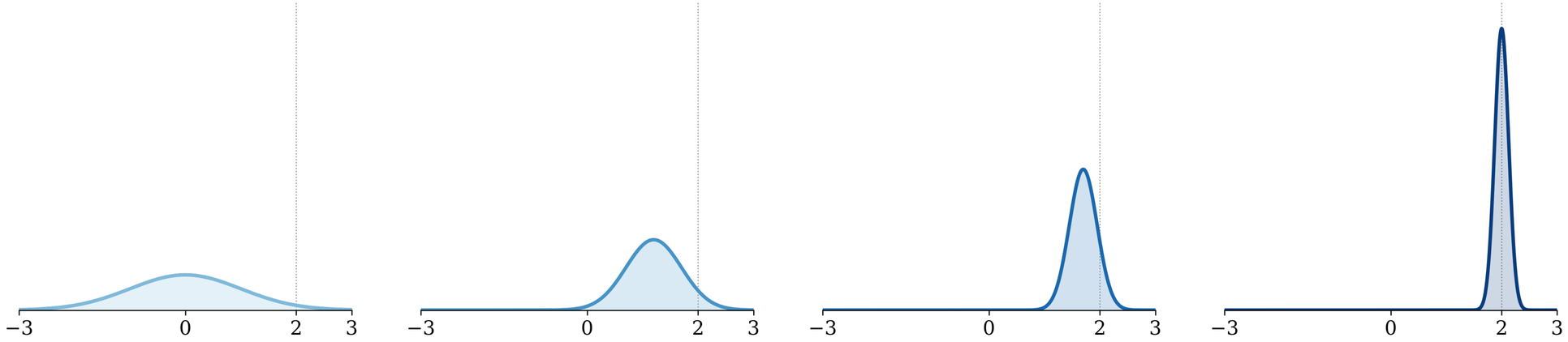
Can we construct a path of distributions $p_t(x_t)$ that interpolates between p_0 and the data distribution?

If the data distribution is a single point x_1, this is easy:

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

Noise schedules: α_t, β_t are such that $\alpha_0 = 0, \alpha_1 = 1 \qquad \beta_0 = 1, \beta_1 = 0$

That is: $x_t = \alpha_t x_1 + \beta_t x_0 \qquad x_0 \sim \mathcal{N}(0,I)$



Flow Matching

Can we construct a path of distributions $p_t(x_t)$ that interpolates between p_0 and the data distribution?

If the data distribution is a single point x_1, this is easy: Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

We can define a marginal path of distributions via:

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

By construction, this interpolates p_0 and p_{data}

We now want to get an ODE-based description of these dynamics

Flow Matching

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

We now want to get an ODE-based description of these dynamics

Recall the continuity equation translates between distributions and samples:

("Local")
Evolution of individual samples

$$dx_t = v(x_t, t)dt$$



("Global")
Evolution of distribution

$$\partial_t p_t = -\operatorname{div}(v_t p_t)$$

Flow Matching

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

We now want to get an ODE-based description of these dynamics

Key idea: find the velocity field generating $p_t(x_t \mid x_1)$ and marginalise it

Conditional velocity field: $v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t \alpha_t) x_1 + \beta_t^{-1}\dot{\beta}_t x$

Flow Matching

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

We now want to get an ODE-based description of these dynamics

Key idea: find the velocity field generating $p_t(x_t \mid x_1)$ and marginalise it

Conditional velocity field: $v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t \alpha_t) x_1 + \beta_t^{-1}\dot{\beta}_t x$

Proof sketch: We want: $dx_t = v_t(x_t \mid x_1)$ when $x_t = \alpha_t x_1 + \beta_t x_0$

Flow Matching

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

We now want to get an ODE-based description of these dynamics

Key idea: find the velocity field generating $p_t(x_t \mid x_1)$ and marginalise it

Conditional velocity field: $v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t \alpha_t) x_1 + \beta_t^{-1}\dot{\beta}_t x$

Proof sketch:

We want: $dx_t = v_t(x_t \mid x_1)$ when $x_t = \alpha_t x_1 + \beta_t x_0$

Equivalently: $\dot{\alpha}_t x_1 + \dot{\beta}_t x_0 = v_t(\alpha_t x_1 + \beta_t x_0 \mid x_1)$

Flow Matching

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

We now want to get an ODE-based description of these dynamics

Key idea: find the velocity field generating $p_t(x_t \mid x_1)$ and marginalise it

Conditional velocity field: $v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t\alpha_t)x_1 + \beta_t^{-1}\dot{\beta}_t x$

Proof sketch: We want: $dx_t = v_t(x_t \mid x_1)$ when $x_t = \alpha_t x_1 + \beta_t x_0$

Equivalently: $\dot{\alpha}_t x_1 + \dot{\beta}_t x_0 = v_t(\alpha_t x_1 + \beta_t x_0 \mid x_1)$

Want $v_t(x \mid u_1)$ for arbitrary x. Reparametrise: if $x = \alpha_t x_1 + \beta_t x_0$ so that $x_0 = \dfrac{x - \alpha_t x_1}{\beta_t}$

Flow Matching

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

We now want to get an ODE-based description of these dynamics

Key idea: find the velocity field generating $p_t(x_t \mid x_1)$ and marginalise it

Conditional velocity field: $v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t\alpha_t)x_1 + \beta_t^{-1}\dot{\beta}_t x$

Proof sketch: We want: $dx_t = v_t(x_t \mid x_1)$ when $x_t = \alpha_t x_1 + \beta_t x_0$

Equivalently: $\dot{\alpha}_t x_1 + \dot{\beta}_t x_0 = v_t(\alpha_t x_1 + \beta_t x_0 \mid x_1)$

Want $v_t(x \mid u_1)$ for arbitrary x. Reparametrise: if $x = \alpha_t x_1 + \beta_t x_0$ so that $x_0 = \dfrac{x - \alpha_t x_1}{\beta_t}$

Hence $v_t(x \mid x_1) = \dot{\alpha}_t x_1 + \dot{\beta}_t \left(\dfrac{x - \alpha_t x_1}{\beta_t} \right) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t\alpha_t)x_1 + \beta_t^{-1}\dot{\beta}_t x$

Check: $p_t(x_t \mid x_1)$ and $v_t(x_t \mid x_1)$ solve the continuity equation

Flow Matching

Define $p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I)$

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

We now want to get an ODE-based description of these dynamics

Key idea: find the velocity field generating $p_t(x_t \mid x_1)$ and marginalise it

Conditional velocity field: $v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t \alpha_t) x_1 + \beta_t^{-1}\dot{\beta}_t x$

The marginal velocity field is then:

$$v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1$$

Interpretation: weighted combination of conditional velocity fields

By Bayes' Rule: weighting factor is $p_t(x_1 \mid x)$, posterior over clean data given noisy data

Equivalently: $v_t(x) = \mathbb{E}\left[v_t(x_t \mid x) \mid x_t = x\right]$

Flow Matching

$$v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1 \qquad p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

Proof sketch: Check that $v_t(x)$ and $p_t(x)$ solve the continuity equation.

Flow Matching

$$v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1 \qquad p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

Proof sketch: Check that $v_t(x)$ and $p_t(x)$ solve the continuity equation.

Plug in for $p_t(x)$ and use the continuity equation for $p_t(x \mid x_1)$:

$$\partial_t p_t(x) = \partial_t \int p_t(x \mid x_1) p_{\text{data}}(x_1) dx_1 = \int -\operatorname{div}\left(p_t(x \mid x_1) v_t(x \mid x_1)\right) p_{\text{data}}(x_1) dx_1$$

Flow Matching

$$v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1 \qquad p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

Proof sketch: Check that $v_t(x)$ and $p_t(x)$ solve the continuity equation.

Plug in for $p_t(x)$ and use the continuity equation for $p_t(x \mid x_1)$:

$$\partial_t p_t(x) = \partial_t \int p_t(x \mid x_1) p_{\text{data}}(x_1) dx_1 = \int -\operatorname{div}\left(p_t(x \mid x_1) v_t(x \mid x_1)\right) p_{\text{data}}(x_1) dx_1$$

$$= -\operatorname{div}\left(\int p_t(x \mid x_1) v_t(x \mid x_1) p_{\text{data}}(x_1) dx_1\right) \qquad \text{(div is linear)}$$

Flow Matching

$$v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1 \qquad p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)]$$

Proof sketch: Check that $v_t(x)$ and $p_t(x)$ solve the continuity equation.

Plug in for $p_t(x)$ and use the continuity equation for $p_t(x \mid x_1)$:

$$\partial_t p_t(x) = \partial_t \int p_t(x \mid x_1) p_{\text{data}}(x_1) dx_1 = \int -\operatorname{div}\left(p_t(x \mid x_1) v_t(x \mid x_1)\right) p_{\text{data}}(x_1) dx_1$$

$$= -\operatorname{div}\left(\int p_t(x \mid x_1) v_t(x \mid x_1) p_{\text{data}}(x_1) dx_1\right) \qquad \text{(div is linear)}$$

$$= -\operatorname{div}\left(p_t(x) \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1\right)$$

$$= -\operatorname{div}\left(p_t(x) v_t(x)\right)$$

Flow Matching: Summary

Conditional dynamics: interpolate p_0 to a single datapoint x_1

$$p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I) \qquad v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t \alpha_t)x_1 + \beta_t^{-1}\dot{\beta}_t x$$

Marginal dynamics: interpolate p_0 to a data distribution p_{data}

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)] \qquad v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1$$

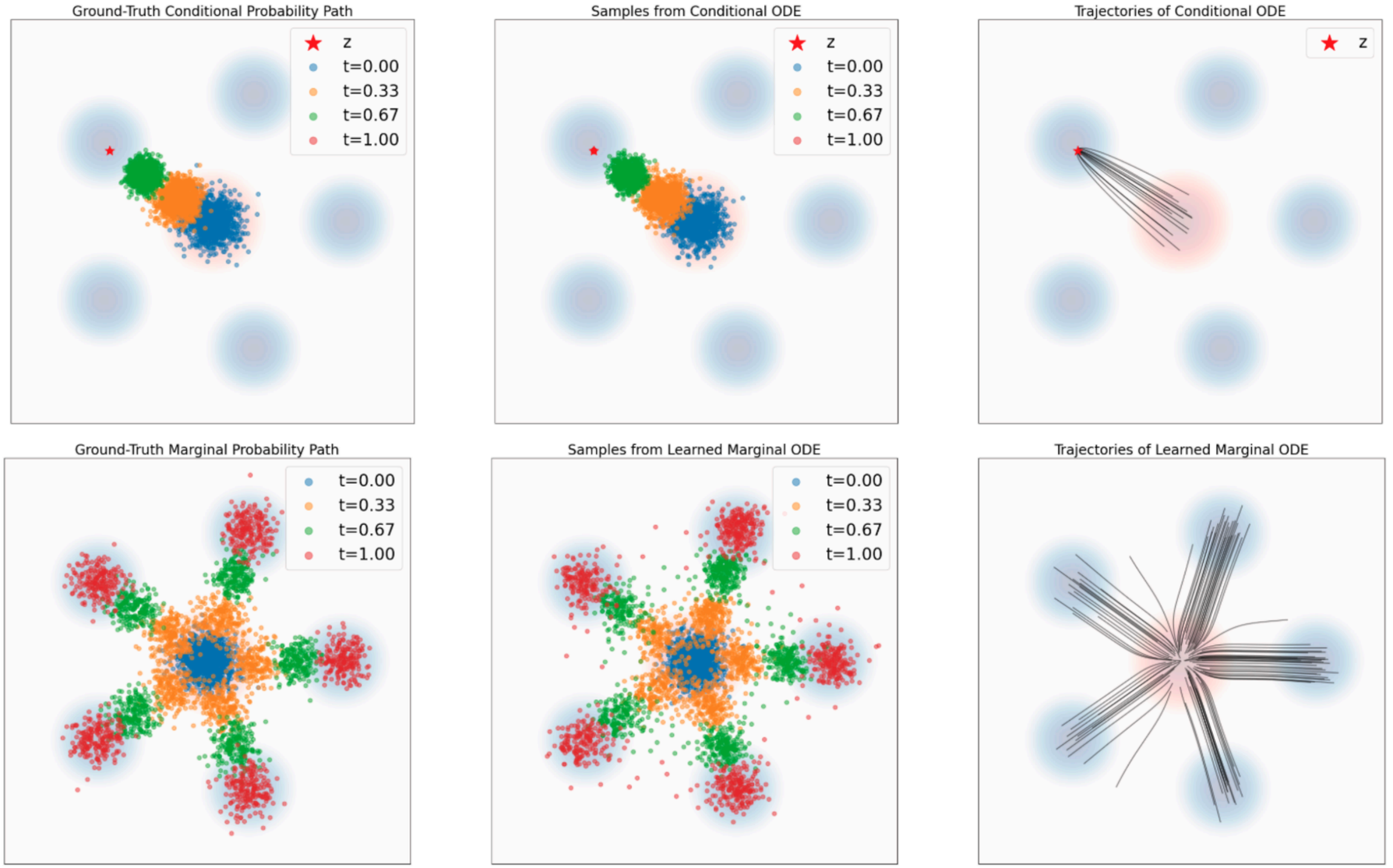
Recall the continuity equation translates between distributions and samples:

("Local")
Evolution of individual samples
$dx_t = v(x_t, t)dt$



("Global")
Evolution of distribution
$\partial_t p_t = -\text{div}(v_t p_t)$

Flow Matching: Illustration



[Holderrieth and Erives, 2026]

Questions?

Flow Matching: Learning

Conditional dynamics: interpolate p_0 to a single datapoint x_1

$$p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I) \qquad v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t \alpha_t)x_1 + \beta_t^{-1}\dot{\beta}_t x$$

Marginal dynamics: interpolate p_0 to a data distribution p_{data}

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)] \qquad v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1$$

If we knew $v_t(x)$, we can sample data via $x_0 \sim \mathcal{N}(0, I)$ and solving $dx_t = v_t(x_t)dt$

Flow Matching: Learning

Conditional dynamics: interpolate p_0 to a single datapoint x_1

$$p_t(x_t \mid x_1) = \mathcal{N}(x_t \mid \alpha_t x_1, \beta_t^2 I) \qquad v_t(x \mid x_1) = (\dot{\alpha}_t - \beta_t^{-1}\dot{\beta}_t \alpha_t)x_1 + \beta_t^{-1}\dot{\beta}_t x$$

Marginal dynamics: interpolate p_0 to a data distribution p_{data}

$$p_t(x_t) = \int p_t(x_t \mid x_1) p_{\text{data}}(x_1) dx_1 = \mathbb{E}_{x_1 \sim p_{\text{data}}}[p_t(x_t \mid x_1)] \qquad v_t(x) = \int v_t(x \mid x_1) \frac{p_t(x \mid x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1$$

If we knew $v_t(x)$, we can sample data via $x_0 \sim \mathcal{N}(0, I)$ and solving $dx_t = v_t(x_t)dt$

But $v_t(x)$ is intractable: requires marginalising over the data and $p_t(x)$ is also intractable

How can we approximate it?

Intuition: should be a regression problem, since $v_t(x)$ is just a vector field

Flow Matching: Learning

Let $v_t^\theta(x)$ be a neural network vector field.

Want to minimise the MSE:

$$\begin{aligned} \mathsf{L}(\theta) &= \mathbb{E}_{t\sim U(0,1)}\mathbb{E}_{x\sim p_t(x)} \quad \|v_t^\theta(x) - v_t(x)\|^2 \\ &= \mathbb{E}_{t\sim U(0,1)}\mathbb{E}_{x_1\sim p_{\text{data}}}\mathbb{E}_{x\sim p_t(x|x_1)} \quad \|v_t^\theta(x) - v_t(x)\|^2 \end{aligned}$$

Flow Matching: Learning

Let $v_t^\theta(x)$ be a neural network vector field.

Want to minimise the MSE:

$$\begin{aligned} \mathsf{L}(\theta) &= \mathbb{E}_{t\sim U(0,1)}\mathbb{E}_{x\sim p_t(x)} \quad \|v_t^\theta(x) - v_t(x)\|^2 \\ &= \mathbb{E}_{t\sim U(0,1)}\mathbb{E}_{x_1\sim p_{\text{data}}}\mathbb{E}_{x\sim p_t(x|x_1)} \quad \|v_t^\theta(x) - v_t(x)\|^2 \end{aligned}$$

Key trick: replace the prediction target with a conditional mean

In general, $\arg\min_f \mathbb{E}_{x,y}\|f(x) - y\|^2 = \arg\min_f \mathbb{E}_x\|f(x) - \mathbb{E}_y[y \mid x]\|^2$

Flow Matching: Learning

Let $v_t^\theta(x)$ be a neural network vector field.

Want to minimise the MSE:

$$\begin{aligned} \mathrm{L}(\theta) &= \mathbb{E}_{t\sim U(0,1)}\mathbb{E}_{x\sim p_t(x)} \quad \|v_t^\theta(x) - v_t(x)\|^2 \\ &= \mathbb{E}_{t\sim U(0,1)}\mathbb{E}_{x_1\sim p_{\text{data}}}\mathbb{E}_{x\sim p_t(x|x_1)} \quad \|v_t^\theta(x) - v_t(x)\|^2 \end{aligned}$$

Key trick: replace the prediction target with a conditional mean

In general, $\arg\min_f \mathbb{E}_{x,y}\|f(x) - y\|^2 = \arg\min_f \mathbb{E}_x\|f(x) - \mathbb{E}_y[y \mid x]\|^2$

Since $v_t(x) = \mathbb{E}\left[v_t(x_t \mid x) \mid x_t = x\right]$, we have that

$$\mathrm{L}(\theta) = \mathbb{E}_{t\sim U(0,1)}\mathbb{E}_{x_1\sim p_{\text{data}}}\mathbb{E}_{x\sim p_t(x|x_1)} \quad \|v_t^\theta(x) - v_t(x \mid x_1)\|^2 + C$$

(Good exercise: actually prove this)

Now tractable, since we have $v_t(x \mid x_1)$ in closed form

Flow Matching: Pseudocode

Popular choice of noise schedules: $\alpha_t = t$, $\beta_t = (1 - t)$

Algorithm 3 Flow Matching Training Procedure (for Gaussian CondOT path $p_t(x|z) = \mathcal{N}(tz, (1-t)^2)$)

Require: A dataset of samples $z \sim p_{\text{data}}$, neural network u_t^θ

1: **for** each mini-batch of data **do**
2: Sample a data example z from the dataset.
3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
5: Set

$$x = tz + (1-t)\epsilon \qquad \text{(General case: } x \sim p_t(\cdot \mid z)\text{)}$$

6: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2 \qquad \text{(General case: } = \|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2\text{)}$$

7: Update $\theta \leftarrow \text{grad_update}(\mathcal{L}(\theta))$.
8: **end for**

[Holderrieth and Erives, 2026]

Stochastic Interpolants

Concurrent work with Flow Matching is Stochastic Interpolants (Albergo et al., 2023)

The two frameworks are essentially the same idea, but a few differences
(in original papers, at least)

Stochastic Interpolants

Concurrent work with Flow Matching is Stochastic Interpolants (Albergo et al., 2023)

The two frameworks are essentially the same idea, but a few differences
(in original papers, at least)

Flow Matching:

Deterministic interpolations

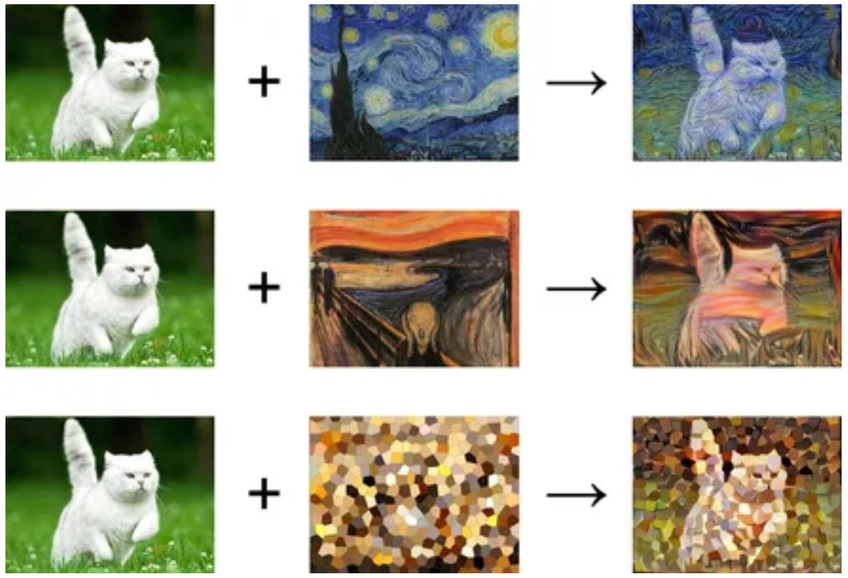
$$x_t = \alpha_t x_0 + \beta_t x_1$$

Stochastic Interpolants:

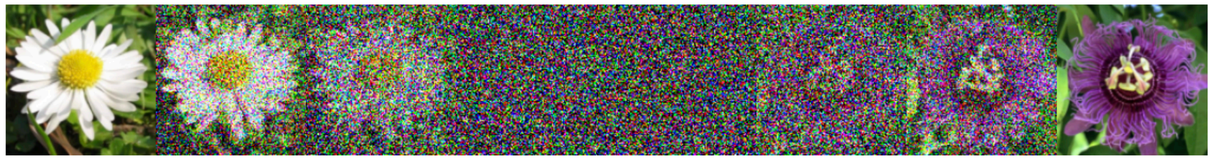
Stochastic interpolations

$$x_t = \alpha_t x_0 + \beta_t x_1 + \gamma_t z$$

z is noise; allows for both x_0, x_1 to be meaningful
(e.g. Style Transfer)



[stateoftheart.ai]



[Albergo et al, 2023]

Stochastic Interpolants

Concurrent work with Flow Matching is Stochastic Interpolants (Albergo et al., 2023)

The two frameworks are essentially the same idea, but a few differences
(in original papers, at least)

Flow Matching:

Deterministic interpolations

ODE-based sampling

Trained with conditional loss

Stochastic Interpolants:

Stochastic interpolations

Allows for ODE or SDE based sampling

Diffusion is a special case

Multiple different possible losses

Summary

Normalising Flows, Diffusion, Flow Matching, Stochastic Interpolants, ….

… are really all instantiations of the same basic idea:

Create a path of distributions from noise to data by learning a velocity field

Each method differs in its details of how this is done

Normalising flows:
- Path is learned, not fixed
- Trained by maximum likelihood at the path endpoint

Diffusion, Flow Matching, Stochastic Interpolants:
- Path is defined a priori
- Learn the corresponding vector field (continuity equation!) via regression

Questions?